CONCURRENCY

Concurrency Model

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

qBittorrent-Fixed's merge service is an **asyncio-first** FastAPI app with a small amount of thread-based plumbing for the legacy download-proxy. This document is the reference for how the pieces fit together, where the hot spots are, and where the pending hardening lives.

The two-thread daemon

`download-proxy/src/main.py` launches two daemon threads in a single Python process:

```
Thread 1: start_original_proxy()   — legacy HTTP server on :7186
Thread 2: start_fastapi_server()   — uvicorn on :7187, own asyncio
loop
Main:     while True: time.sleep(60)
```

The main thread keeps the process alive; `KeyboardInterrupt` logs and exits. **Graceful shutdown is pending Phase 3** — today a `SIGTERM` races the two threads and can drop in-flight requests. See `architecture/shutdown-sequence.mmd` for the target sequence.

Reference: `download-proxy/src/main.py:69-83`.

Search fan-out

The hot path. `SearchOrchestrator.search()` in `download-proxy/src/merge_service/search.py:270` fans out one coroutine per tracker:

```
search_results = await asyncio.gather(*[_search_one(t) for t in
        trackers])
```

(`search.py:308`).

Characteristics:

- **~40 trackers** run **fully concurrently** today — no Semaphore cap.
- Each `_search_one(t)` stores its results incrementally in `self._tracker_results[search_id][tracker.name]` so the SSE stream can surface partial results before `gather()` returns.
- `asyncio.gather` propagates exceptions unless each task catches its own; the code does catch per-tracker so one failing tracker does not abort the merge.

Pending hardening (Phase 3):

- Introduce `asyncio.Semaphore(MAX_CONCURRENT_TRACKERS)` — env var `MAX_CONCURRENT_TRACKERS`, default **5**. Status: **pending**.
- Wrap each per-tracker coroutine in `tenacity.AsyncRetrying` for exponential back-off (currently a failed tracker returns `[]` immediately).
- Add `pybreaker.CircuitBreaker` per tracker so a misbehaving tracker is fast-failed for a cool-down window instead of slowing the whole search on every query.

Subprocess-based plugin execution

Public trackers run out-of-process via `asyncio.create_subprocess_exec("python3", "-c", <script>, …)` with a **10-second hard timeout** (`search.py:407`). The subprocess gets its own `asyncio.subprocess.Pipe` for stdout/stderr. If it times out:

```
await asyncio.wait_for(proc.communicate(), timeout=10)
...
if proc and proc.returncode is None:
    proc.kill()
    await proc.wait()
```

Phase 2 replaces this with an in-process `ProcessPoolExecutor` that keeps one warm worker per tracker and enforces resource limits.

TTL caches for ephemeral dicts

Several dicts grow unbounded today:

Dict	File	Growth hazard
`_active_searches`	`search.py:233`	One entry per search forever
`_last_merged_results`	`search.py:235`	One tuple per search forever
`_tracker_results`	`search.py:236`	

Dict	File	Growth hazard
		One nested dict per search forever
`_tracker_sessions`	`search.py:234`	One entry per authenticated tracker; overwritten, not leaked, but aiohttp sessions never close
`_pending_captchas`	`api/auth.py:24`	Grows on every `/captcha` call
`_execution_logs`	`api/hooks.py:56`	Capped at 200 via `LOGS_MAX` (good)
`streaming seen_hashes`	`api/streaming.py`	Per-SSE-generator set, mutated concurrently

Pending hardening (Phase 3):

- Replace each dict with `cachetools.TTLCache(maxsize=..., ttl=...)`.
- Close `aiohttp.ClientSession` at the end of each tracker search (currently reused across calls; at minimum add a `close()` on orchestrator shutdown).
- Lock `_execution_logs` append with `asyncio.Lock` — currently a data race if two hooks finish concurrently.

Retry + circuit-breaker

Today: zero use of `tenacity`, `backoff`, or `async-timeout` anywhere in the code. A single request failure surfaces as a zero-result tracker.

Phase 3 target:

- `tenacity.AsyncRetrying` wrapping every outbound HTTP call (aiohttp + subprocess). Exponential back-off with jitter, max 3 attempts.
- `pybreaker.CircuitBreaker` per tracker, opening after 5 consecutive failures and closing after a 60 s recovery probe.
- Both pieces share a single `TrackerHealth` dataclass so `/api/v1/trackers/health` can expose the current state.

Streaming (SSE)

`download-proxy/src/api/streaming.py` (222 lines) owns the live `/api/v1/search/stream/{search_id}` endpoint. Behaviour:

- Each client gets its own generator with a local `seen_hashes: set()`.
- The generator polls `orchestrator.get_live_results(search_id)` in a `while True: await asyncio.sleep(0.5)` loop.
- Deduplication happens client-side (`seen_hashes`).
- **Gap:** no disconnect detection — the generator keeps polling even after the HTTP client drops. Phase 3 adds a `await request.is_disconnected()` check at the top of the loop.

Graceful shutdown

Target sequence (Phase 3 target, not yet implemented):

1. `SIGTERM` received by the container.
2. Signal handler sets a global `_shutting_down` flag.
3. FastAPI rejects new requests (503) and cancels in-flight search tasks via a shared `asyncio.Event`.
4. `aiohttp.ClientSession` instances close.
5. `asyncio.Task`s are awaited with a 5 s grace period.
6. Daemon threads are joined with `join(timeout=5)`.
7. Process exits 0.

See `architecture/shutdown-sequence.mmd` for the target sequence diagram.

Concurrency tests

- **Unit** — `tests/unit/test_merge_trackers.py` exercises the orchestrator happy path.
- **Performance** — `tests/performance/test_concurrent_search.py` benchmarks the gather fan-out.
- **Stress** — `tests/stress/test_search_stress.py` runs the orchestrator at elevated load.
- **Benchmarks** — `tests/benchmark/test_search_benchmark.py` and `test_deduplication_benchmark.py` track p50/p95 regressions.
- **Pending (Phase 3)** — `tests/concurrency/` uses pytest-randomly + pytest-repeat to surface asyncio race conditions.

Observability hooks

Once the Prometheus/Grafana stack is up (see `OBSERVABILITY.md`), the following metrics surface the concurrency health:

- `qbit_merge_active_searches` — gauge
- `qbit_merge_tracker_requests_total` — counter, labelled by tracker
- `qbit_merge_search_duration_seconds_bucket` — histogram
- `qbit_merge_circuit_breaker_state` — gauge (0 = closed, 1 = half, 2 = open), labelled by tracker

References

- Fan-out site — `download-proxy/src/merge_service/search.py:308`
- Daemon threads — `download-proxy/src/main.py:69-83`
- SSE generator — `download-proxy/src/api/streaming.py`
- In-memory caches — `download-proxy/src/merge_service/search.py`, `download-proxy/src/api/auth.py`, `download-proxy/src/api/hooks.py`
- Sequence diagram — `architecture/shutdown-sequence.mmd`